QCon
全球软件开发大会

InfoQ 技术大会

如何构建 AI Native 的数据语义层

从 Data Agent 实践到可执行的 Semantic Runtime

赵恒

Datus AI

QCon 全球软件开发大会 · 2026



QCon
全球软件开发大会

InfoQ 技术大会



目录

01 从「查出指标」到「解释变化」
传统 Semantic Layer 在 Agent 场景中的四个断点

02 构建可执行的 Semantic Runtime
一个轻量 SQL 编译器的工程实现

03 让语义层进入 Reasoning Loop
Semantic Tools 与 Ontology Tools 如何设计

04 端到端案例复盘
从业务问题到可验证的分析结果

01

从「查出指标」到「解释变化」

传统 Semantic Layer 在 Agent 场景中的不足

一年 Data Engineering Agent 实践：Day 1 就有语义层

统一口径不是后补的治理，是 Agent 查询准确率的前提， 数据工程不再是给人交付表，而是给Agent交付语义层

https://github.com/Datus-ai/Datus-agent/

Day 1 任务升级 语义层重构

Agent + Semantic Layer

开源 Data Engineering Agent 上线即接入语义层；指标口径统一后，NL2Metrics 的准确率明显高于 NL2SQL

查指标 → 解释变化 → 指导行动

用户不再满足于「本月收入多少」，而是「收入为什么下降」——任务从一次查询变成一条分析链路，进而在语义层之上构建本体，绑定业务对象与可执行操作

MetricFlow → Ossie + Dosi

现有实现与表达能力撑不住分析链路；转向开放规范 + 自研可执行的 Semantic Runtime

Agent 不会消除口径漂移，只会放大它

同一个问题，人写 1 条 SQL，Agent 一次会话里发起 6–11 次工具调用、写出多条；每条都在重新决定 join、过滤与粒度。语义层的价值不是Agent的另一个prompt，而是「每一条都用同一份定义」，让Agent的SQL更稳定

语义层解决了什么：每个人都在重新定义 revenue

同一个问题、三个消费者、三个「收入」——都能跑通，都声称自己是对的

没有语义层 · 各写各的 SQL

BI 看板

```
SUM(amount)
```

$1.24M

分析师

```
SUM(amount)
  WHERE status != 'cancelled'
```

$1.19M

AI Agent

```
SUM(amount * fx_rate)
```

$1.31M

有语义层 · 一个定义，一次编译

```
metrics:
- name: revenue
  expression: SUM(orders.amount)
  filter: orders.status
          != 'cancelled'
```

$1.19M

看板、分析师、Agent 拿到同一个数；换 BI 工具或换 Agent，数不变

语义层的第一性价值：让每个消费者「要指标，而不是要 SQL」—— 这一点在 Day 1 就成立，也是后面所有问题的起点

拆解一个分析任务：「收入为什么下降？」

从一次指标查询，走到维度拆解、订单明细与原因验证——这是全场的地图

1 趋势查询	2 维度拆解	3 明细检查	4 归因	5 验证
revenue 按季度，Q1 → Q2 −18.0%	按 region / channel / category 分别看	West 区取消订单到底长什么样	哪个维度解释了多少下降	拆解加得回总数？假设站得住？
1 次调用	2–3 次	1–2 次	1 次	1–2 次

断点① 指标与明细脱节

下钻时丢口径、换粒度

断点② 多表关联重复计数

拆维度时 join 扇出

断点③ 衍生指标膨胀

每个新口径都成了新指标

断点④ 失败后反复试 SQL

错误信息不可行动

一个问题 = 6–9 次工具调用；每一跳都要求同一份语义——而传统语义层只覆盖第 1 跳

断点一：指标查对了，下钻就错了

Agent 从聚合指标进入明细时，口径在手写 SQL 里悄悄消失、粒度悄悄改变

第 1 跳 · 语义层编译

```sql
-- query revenue by region
SELECT c.region, SUM(o.amount) AS revenue
FROM orders o JOIN customers c
  ON o.customer_id = c.customer_id
WHERE o.status != 'cancelled'
GROUP BY c.region
```

第 3 跳 · Agent 手写明细

```sql
-- "look at West orders"
SELECT o.order_id, oi.product_id, oi.amount
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
JOIN customers c ON ...
WHERE c.region = 'West'    -- 过滤没了，粒度变了
```

结果合理，但实际错误

1. **口径丢失**

   cancelled 过滤只存在于指标定义里，明细 SQL 没有继承

2. **粒度漂移**

   revenue 在订单粒度，明细查到了订单行；加总回去数字对不上

3. **无人察觉**

   两个结果单独看都「说得通」，差异只在对账时才暴露

根因：语义层只服务「聚合平面」，明细探索被交还给了手写 SQL

断点二：SQL 跑通了，数字错了

三类错误都能执行、量级都对、一眼看不出——拆维度时最容易发生

JOIN 扇出

120 → 360

delay_minutes 在延误粒度；按 route 拆维度时 join 到 flights 复制了 3 次。SUM 悄悄乘了 3。

✓ 能执行 ✓ 量级正确 ✗ 结果错误

比率不可加

AVG(AOV) ≠ AOV

avg_order_value = revenue / orders。对各区的 AOV 再求平均，得到一个看起来正常、实际错误的数。

✓ 能执行 ✓ 量级正确 ✗ 结果错误

粒度错配

margin = revenue − cost

两个事实表粒度不同，一次 join 同时算两边，两边都被放大、两边都错。

✓ 能执行 ✓ 量级正确 ✗ 结果错误

这不是 Agent「不够聪明」：fan-out、比率与粒度是关系代数问题，应由引擎证明，而不是由 prompt 提醒

断点三：衍生指标膨胀

时间窗口 × 筛选条件 × 派生计算不断组合，每个口径都变成一个新指标

指标爆炸 · 每个过滤都成了一个指标

- US GMV
- US iOS GMV
- US iOS 新用户 GMV
- US iOS 新用户 付费渠道 GMV
- retention_7d · retention_14d · retention_30d
- revenue_mom · revenue_yoy · revenue_ytd · revenue_t7d ...

6 → 几千

六个稳定定义覆盖电商主干；平台里却沉淀了几千个离散指标

对 Agent 的后果

1. **选不准**
   RAG 在几千个相似名字里检索，top-k 常常全是近义指标
2. **口径冲突**
   同名不同义、同义不同名；两个指标的差异藏在 SQL 里
3. **无法组合**
   「上月 West 区新用户 7 日留存」不是任何一个现成指标
4. **无法维护**
   基础口径一改，几百个衍生指标各自漂移

需要的不是更大的指标目录，而是更少、可组合的定义——第二段会给出做法

断点四：失败之后反复试 SQL

模糊的错误信息 + 没有约束的接口 = Agent 原样重试，直到耗尽预算

Agent 看到的错误

```
Traceback (most recent call last):
  File ".../metricflow/engine/metricflow_engine.py", line 612, in query
    return self._create_execution_plan(mf_query_request)
  File ".../dataflow/builder/dataflow_plan_builder.py", line 1203, in ...
    raise UnableToSatisfyQueryError(
metricflow.errors.errors.UnableToSatisfyQueryError:
  Recipe not found for measure spec: MeasureSpec(element_name='revenu')
  ... (47 more lines)
```

哪个名字错了？正确的候选有哪些？下一步该改什么？—— 堆栈里一个都没有。

重试循环

生成 SQL / 查询

↓

执行失败

↓

解析堆栈，猜原因

↓

原样再试 ×N

3–5 轮

一次拼写错误引发的典型重试轮数；每轮都是一次完整编译 + 执行

Agent 需要的是「可行动的错误」：稳定的错误码、候选名字、修正建议——第三段展开

从 MetricFlow 到 Apache Ossie：为什么换指标定义规范

把三类原因分开说：实现的限制、表达的不足、以及对开放标准的需求

实现的限制

- Python CLI，每个查询一个进程，冷启动 2.9–3.1 s
- 错误是堆栈而非合同
- Agent 循环里每一步都是一次编译，成本被放大 6–11 倍

表达的不足

- 窗口 / 同环比 / 参数化指标只能手写成离散指标
- 明细探索（select）不在语义模型表达范围内
- 业务对象与关系（ontology）没有位置

开放标准的需求

- 定义应是你自己拥有的 YAML，可进 PR、可 diff
- Snowflake 发起、走向 Apache 治理；dbt / Databricks / Tableau 双向转换器
- Iceberg 让表可移植，Ossie 让指标可移植

SQL 标准：数据库引擎 ≈ Apache Ossie：Dosi

规范负责「指标是什么意思」；引擎负责「怎样编译成正确的、特定方言的 SQL」。规范是可移植的，但还不可执行——这个缺口就是第二段。

02

构建可执行的 Semantic Runtime

一个轻量 SQL 编译器的工程实现

从语义模型到 SQL: 一条稳定的SQL生成旅程

编译期只做一件事——把语义翻译成特定方言的正确 SQL；没有中间存储，没有第二份语义

OSI YAML
`dosi-model`
datasets · relationships
metrics · dimensions

Semantic IR
`dosi-compiler`
解析 + 校验
类型化的语义单元

逻辑计划
`dosi-planner`
关联路径选择
聚合与粒度规划

SQL AST
`polyglot-sql`
方言无关的
查询树

方言 SQL
`16+ dialects`
DuckD
B · TiDB · Snowflake
ClickHouse · BigQuery …

命令

```
dosi query --model orders/model.yaml \
  --metrics revenue,order_count \
  --group-by orders.order_date:month \
  --dialect tidb --pretty
```

Agent 只给出指标、维度、时间粒度与方言；它从不输入 SUM()。

编译产物 · TiDB

```
SELECT
  STR_TO_DATE(DATE_FORMAT(orders.order_date,'%Y-%m-01'),
              '%Y-%m-%d')            AS order_date__month,
  SUM(orders.amount)                 AS revenue,
  COUNT(orders.order_id)             AS order_count
FROM main.orders AS orders
GROUP BY 1
```

换 ClickHouse 就是 toStartOfMonth —— YAML 一行不动，正面回收「方言漂移」

编译而不是检索：语义不是塞进 prompt 的上下文，而是经过类型检查、能被规划、能被证明的 IR

语义模型解析与校验：错误尽量在编译期暴露

主键、关系基数、表达式类型、时间粒度——每一项都是后面规划正确性的前提

```
semantic_model:
- name: orders
  datasets:
  - name: orders
    primary_key: [order_id]        # 粒度
  - name: customers
    primary_key: [customer_id]
  relationships:
  - from: orders   to: customers
    from_columns: [customer_id]    # N:1
  metrics:
  - name: revenue
    expression: SUM(orders.amount)
    filter: orders.status != 'cancelled'
  - name: avg_order_value
    expression: revenue / order_count   # 比率
```

1. **主键 → 粒度**
   每个 dataset 的行由什么唯一标识；这是后面所有「在哪个粒度聚合」判断的根

2. **关系 → 基数**
   N:1 可安全展开，1:N 会复制行；基数未知时规划器宁可拒绝

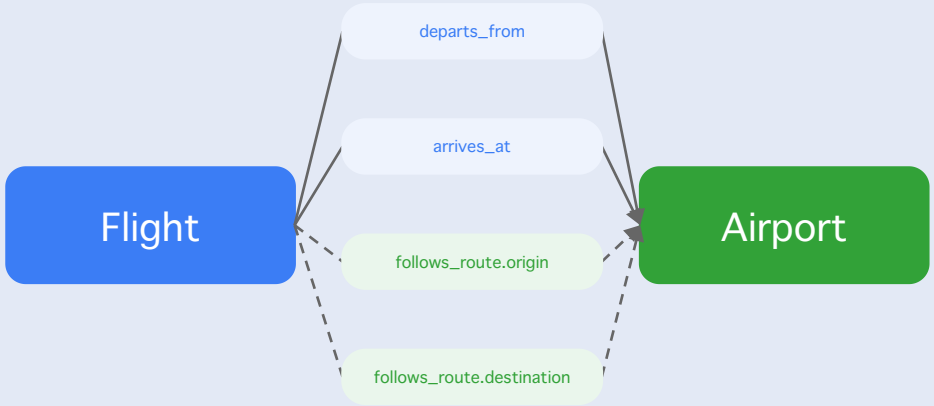
3. **表达式 → 类型**
   SUM / COUNT / AVG / COUNT DISTINCT / 比率；类型决定该指标能否求和、上卷、归因

4. **时间 → 粒度与窗口**
   时间维度的最小粒度、日历与窗口语义，是同环比正确性的前提

dosi validate 可运行上游 apache/ossie 参考校验器：对规范校验，而不是对某个引擎的理解校验

关联路径选择：歧义是错误，不是猜测

两个数据集之间若有多条路径，规划器列出全部候选，而不是默默选一条



四条路径，两条是直接外键，两条经由 Route；其中 departs_from ≡ follows_route.origin（本体可以声明等价）

Agent 问：flights 按 airports.name 看延误

```
{
  "code": "ambiguous_join_path",
  "message": "4 paths from flights to airports",
  "candidates": [
    "flights_departs_from",
    "flights_arrives_at",
    "flights_follows_route.origin",
    "flights_follows_route.destination"
  ],
  "suggested_retry": "name the relationship
     explicitly, e.g. via=flights_arrives_at"
}
```

候选是完整的、可以原样贴回下一次调用的名字

路径唯一性是正确性的一部分：少一个 join 条件就多一倍行数，规划器不替 Agent 下这个赌注

聚合规划：粒度与 fan-out 的三条出路

引擎不会发出一条它无法证明安全的、对重复行敏感的聚合；三条路，只走一条

路径 1

单粒度直接聚合

请求的维度全部位于一条无行复制的关联路径上——直接 GROUP BY。

路径 2

按粒度拆分 → 合并

每个指标在自己的事实粒度聚合成 CTE；Fact 与 Fact 之间先对齐粒度，再合并（见右）。

路径 3

拒绝：fan_out_risk

两条都证明不了——返回结构化错误，点名出问题的关系，并给出可改查的更细粒度。

路径 2 的产物 · 两个指标、两个粒度、一次请求

```
-- total_delay_minutes (Delay 粒度) + passengers (Flight 粒度)
WITH m0 AS (
  SELECT aircraft.tail_number,
         SUM(delays.delay_minutes) AS delay_min
  FROM delays LEFT JOIN flights … LEFT JOIN aircraft …
  GROUP BY aircraft.tail_number),
m1 AS (
  SELECT aircraft.tail_number,
         SUM(flights.passenger_count) AS pax
  FROM flights LEFT JOIN aircraft …
  GROUP BY aircraft.tail_number)
SELECT COALESCE(m0.tail_number, m1.tail_number),
       m0.delay_min, m1.pax
FROM m0 FULL OUTER JOIN m1
  ON m0.tail_number = m1.tail_number
-- 本例中 passengers 不会被延误行数放大
-- 粒度已一致（均为 aircraft.tail_number）：直接 FULL OUTER JOIN
-- 粒度不一致：D-conform 对齐后合并；更复杂的场景：semi join
```

生成的 SQL 对仓库友好：一次请求一条有界的 pushdown 查询，交给仓库优化器与物化视图；语义层无缓存、无存储

指标代数：这个指标能不能这样用？

可加 / 可上卷 / 可重构 / 不透明——从 IR 推导，作者零额外字段，Agent 聚合前先问引擎

Additive 可加

SUM · COUNT

跨分组可直接相加

Rollupable 可上卷

MIN · MAX

可上卷，但不可相加

Reconstructible 可重构

AVG

分解为 sum / count 后再合成

Opaque 不透明

COUNT(DISTINCT)

两者都不行；必须回到叶子重算

推导出的四种能力 · 每个指标

summable	跨分组可求和 → 维度拆解时可直接加总
rollup-safe	可从细粒度上卷到粗粒度，不需重算
attributable	可归因，并由类型决定归因方法
forecastable	可按时间序列外推

通过 dosi list metrics 与 Python / MCP 暴露为 attributable / attribution_strategy 字段

Agent 在聚合、下钻、解释之前，先问引擎「这个指标支持吗」——答案来自语义模型，不来自 prompt

我们不需要几千个指标

参数化指标、派生表达式与窗口语义组合成一棵可追溯的指标树，而不是几千个离散指标 + RAG

Datus 扩展 · 一个定义，N 个绑定

```
metrics:
- name: trailing_revenue
  base: revenue
  rolling: {function: sum, periods: {param: n}}
- name: revenue_yoy
  base: revenue
  pop: {offset: 12 periods, calculation: percent_change}
- name: revenue_ytd
  base: revenue
  cumulative: {reset: year}

# retention_7d / 14d / 30d  =  retention(n)，一个定义
```

指标爆炸

- US GMV
- US iOS GMV
- US iOS 新用户 GMV
- retention_7d / 14d / 30d
- ……几千个

语义压缩

- GMV · Orders · Buyers
- AOV · Conversion · Retention
- + 维度过滤
- + 派生 / 组合
- + 参数绑定

命名指标是契约，维度过滤是参数

实测 · 五个滚动窗口（7/14/30/60/90 天）一次请求，1M 行

Dosi	MetricFlow	Cube
12 ms	220 ms	1,920 ms
1 个参数化指标 × 5 绑定 → 1 个 base CTE + 5 个 OVER 子句，0.9 KB SQL	5 个手写 cumulative 指标 → 5 次 time-spine join，5.1 KB SQL	5 个手写 rolling measure → 同一个日期序列 join 5 次，6.8 KB SQL

面向 Agent 高频调用的工程取舍

BI 时代，一个看板 / 一次问数只调用一次语义层，延迟无关紧要；Agent 循环里，一条推理链要多轮调用

用户提问 | Agent 推理 | list / describe | query / select | attribute | 仓库执行

一个问题 → 6–11 次工具调用 → 每次都是一次编译。编译成本要乘以循环次数。

	Dosi	MetricFlow	Cube
服务冷启动 → 首个查询	60 ms	~3 s	2.2 s
热编译（未见过的查询）	< 1 ms	—	12–28 ms
每请求引擎开销（1M 行）	~0–4 ms	~3 s	13–22 ms
五个窗口一次请求（SQL 执行）	12 ms	220 ms	1,920 ms
并发 16，滚动窗口吞吐	67 q/s	1.0 q/s	3.1 q/s
200 数据集 / 3,000 指标冷启动	0.49 s	6.7 s	4.7 s

dosi 0.1.12 · dbt-metricflow 0.15.0 · Cube 1.8.4 · 同一语义模型、同一 DuckDB（1M 行）、24 条查询，结果先断言一致（24/24）再计时。引擎开销 = 服务端请求 − 同 SQL 直连 DuckDB。MetricFlow 无服务端，每次查询独立进程，冷启动即每次成本。看比例，不看毫秒。

职责划分

- 语义层：解析、校验、路径与聚合规划、方言生成
- 数据库：扫描、join、聚合、物化视图——它本来就擅长的事
- 不做缓存、不做二级存储：一次请求一条 pushdown SQL
- 语义模型复用：加载一次、常驻内存，每次查询只付编译成本

仓库那一半的活儿三家跑得差不多；差距在生成的 SQL 结构和引擎开销——而这两样在 Agent 循环里被放大了 6–11 倍

03

让语义层进入 Reasoning Loop

Semantic Tools 与 Ontology Tools 如何设计

一个问题 = 6–11 次调用：工具如何覆盖完整分析过程

回到第一段的五步地图，把每一步映射到一组工具——发现、编译、执行、校验

1 趋势查询 → 2 维度拆解 → 3 明细检查 → 4 归因 → 5 验证

discover

有什么？

- list_datasets
- list_metrics
- list_dimensions
- describe_metric
- get_capabilities

compile

会生成什么 SQL？

- compile_sql
- explain
- compile_select
- explain_select

execute

跑出来是什么？

- query
- select
- attribute

validate

语义模型本身对不对？

- validate_model

步骤 1–2 → query · attribute | 步骤 3 → select | 步骤 4 → attribute | 步骤 5 → explain + query 对账

14 个工具，一个编译器、一个归因引擎、一个错误合同

CLI = REST/Arrow = MCP = Python：不是四套实现，是一个引擎的四扇门。服务端随每个响应附带 agent instructions，没见过 Dosi 的大模型也知道怎么循环。

工具接口：输入是语义，输出带证据，边界是特性

Agent 给出指标、维度、过滤与时间范围；拿回结果、生成的 SQL 和所用的定义——但语义模型之外它仍然要写 SQL

引擎负责

凡是已经有定义的

- 指标的含义：过滤、粒度、币种、取消
- 数据集之间的关联路径——声明一次，不再猜
- 聚合顺序：比率、扇出、跨表表达式
- 仓库方言——同一份 YAML，16+ 种 SQL

调用合同

输入

- metrics[] 维度无关的指标名
- dimensions[] 含时间粒度
- filters[] 维度 = 值 / 范围
- time_range 窗口与日历
- dialect 目标方言

输出

- rows Arrow / JSON
- sql 实际执行的语句
- definitions 用到的指标与关系
- error code · candidates · retry

Agent 仍然写 SQL

语义模型建好之前，以及语义模型之外

- 探索还没人建模的原始表
- 规范表达力之外的问题：一次性逻辑、临时 join
- 把候选指标做成原型，再提升进语义模型

这些查询正是新定义的来源：语义层是它们值得保留时的落点。

语义层让 AI 写更少的 SQL，而不是不写：有定义的部分确定性编译，没定义的部分开放探索——边界本身是特性

为什么要在 Metrics Query 与 Detail Select 之间切换

聚合平面与明细平面共用同一套语义：同样的关联关系、同样的过滤、同样的业务定义

聚合平面 · query

```
query(metrics=[revenue],
      dimensions=[customers.region],
      time=Q2-2024)
```

→ West $1.14M（−$1.25M vs Q1）

明细平面 · select

```
select(fields=[orders.order_id, orders.amount,
               customers.region, orders.status],
       filters=[customers.region='West'], limit=50)
```

→ 50 条订单，status 列里 cancelled 的比例一眼可见

同一个语义模型

orders → customers 路径 | cancelled 过滤 | 订单粒度 primary_key | 方言与时间日历

口径继承

select 的过滤与 query 的指标过滤来自同一份定义；想看「计入 revenue 的订单」就带上它，想看全部也行——但那是显式选择

粒度明确

select 的粒度由 dataset 主键决定，加总回去与 query 对得上

名字可贴回

select 里出现的每个字段名，都能原样放进下一次 query 的 dimensions 或 filters

从指标到业务对象的连续分析

指标 → 归因 → 明细 → 对象 → 关系 → 另一个指标：沿同一套定义走完整条链路，不在任何一跳重写口径

指标	归因	明细	对象	关系	指标
revenue	attribute	orders	Customer	acquired_via → Channel	cancel_rate
Q2 −18.0%	West 贡献 71.5%	取消订单集中在 West	新客占比异常高	一个付费渠道	取消率 8.2% → 27.9%
`query`	`attribute`	`select`	`describe(Customer)`	`explain(Customer→Channel)`	`query by channel`

回到总体：拆解加得回 −$1.75M 吗？

没有本体时会发生什么

- 「Customer」在语义模型里只是 customers 表和一个外键列
- Customer → Channel 走哪条 join，agent 得猜
- 每换一个对象，口径与粒度都要重新手写一遍

有本体时

- 对象、属性、关系是可发现、可遍历的一等公民
- 关系有名字、有方向、有多重性，路径不用猜
- 指标与对象绑定：从对象能找到它的指标，反之亦然

连续分析的本质：每一跳都沿用同一份业务定义与关联关系——这正是本体要在查询执行里扮演的角色

Ontology 如何参与查询执行：lowering 到语义模型

Objects / Properties / Relationships 映射到 datasets / keys / relationships——SQL 一行不变，变的是 Agent 看到什么

```
# ontology.yaml
concepts:
- name: Flight
  identify_by: [id]
  properties: [carrier, status]
  relationships:
  - name: departs_from  to: Airport  role: origin
  - name: arrives_at    to: Airport  role: destination
value_domains:
  AlertStatus: [OPEN, RESOLVED]
rules:
  - departs_from ≡ follows_route.origin
```

lowering

Dosi · 加载期

- Object → dataset + primary key
- Property → dimension
- Relationship → relationship + 角色名
- Value domain → 编译期校验
- Rule → 路径等价表

一个普通的语义模型

+

一张概念路径名表

```
Flight.arrives_at.name
→ airports.name via
  flights_arrives_at
```

值域与规则

有界的取值空间：'UNRESOLVED' 在编译期就被拒绝

确定性路径

四条平行边有名字、有角色、有等价声明——不再 ambiguous

跨语义模型的链接

一个语义模型是一个角色的局部理解边界；本体把边界之间连起来

本体不换规划器、不换 SQL；它只改变 Agent 在写查询之前能看到什么、能确定什么

describe / explain / 概念路径：三个新增的工具面

它们打印出的每一个名字，都能原样贴进 select 或 query——这就是合同

describe () · describe(concept)

目录，然后是一个概念的打包视图：属性及类型与值域、出入链接及多重性与角色、可达指标及粒度、可达时间维度。约 300–500 token。外键不作为属性出现。

消除：根表 · 字面量 · 扇出方向

path({from, to})

两个概念之间所有命名路径，带跳数、角色、多重性，以及本体声明的等价关系。把今天的「ambiguous_join_path 失败再重试」变成一次有意的查找。

消除：四条平行边选哪条

select / query with concept paths

FlightAlert.flight.arrives_at.name 解析到与 dataset 写法相同的字段；输出列名可预测。新增编译期拒绝 value_not_in_domain，附候选。

消除：静默的零行结果

3 → 9 → 0

3 个用户问题，9 次工具调用，0 次猜测物理层

< 2K tokens

整个循环的元数据：按概念懒加载，从不整库 dump

2 → 0

两个错误答案陷阱（坏字面量、错机场边）变成查找

3 / 3

今天的规划器生成的 SQL 一条未改——本体只贡献了名字

在航空 fixture（flights · delays · alerts · sensors）上端到端走通。Dosi 仍是编译器：不做搜索、不做权限、不做存储——那些属于 Agent 层。

从失败中恢复：结构化错误合同

堆栈对大模型是噪声；稳定的错误码、候选名字和修正提示是指令——错误合同是 API 的一部分

```
// 未知指标 — 近似匹配
{
  "code": "unknown_metric",
  "message": "metric 'revenu' is not defined",
  "candidates": ["revenue", "revenue_yoy",
                 "order_count"],
  "suggested_retry": "use one of the candidates"
}

// 扇出风险 — 拒绝而非猜测
{
  "code": "fan_out_risk",
  "relationship": "orders_to_order_items",
  "suggested_retry": "query revenue at orders grain,
     or use order_items.line_amount instead"
}
```

合同：每个引擎失败带三个字段

code	稳定标识符，Agent 可据此分支
candidates	名字没匹配上时，给出合法名字
suggested_retry	形状问题的修正提示，一句话

四类错误码

fan_out_risk	ambiguous_join_path
unknown_metric	value_not_in_domain

「Agent 应该读错误并修正查询，而不是原样重试」—— 引擎宁可不返回数字，也不返回错数字，并用大模型能行动的形式说明原因

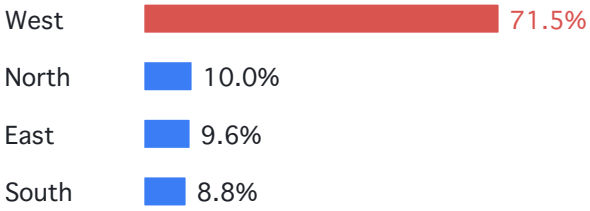
归因作为一个工具：方法随指标类型，结果对总量对账

attribute 由引擎计算分解、由 Agent 负责解释；对可归因的指标，分解对总量对账，解释可检查而非即兴

```
dosi attribute --metric revenue \
  --dimensions customers.region,products.category \
  --baseline 2024-01-01..2024-04-01 \
  --current  2024-04-01..2024-07-01
```

Q1 → Q2 2024

$9.72M → $7.97M　　-$1.75M（-18.0%）



West 内部：取消率 8.2% → 27.9%，附下钻 SQL 供下一步

方法随指标类型

term_wise　**可加指标**

按段精确拆分，各段之和 = 总变化

mix_shift　**比率**

结构（权重变化）vs 表现（分段率变化）

factor_shapley　**乘积与嵌套比率**

对叶子因子做 Shapley，公平分摊交互项

*方法由 P14 的指标代数自动选择：*attribution_strategy *字段*

LLM 自己归因时百分比加不到 100；这里引擎计算、Agent 解释——分解对总量对账，解释因此可检查

04

端到端案例复盘

从业务问题到可验证的分析结果

构建案例所需的语义上下文

从已有的 Schema、历史 SQL 和业务文档里抽取语义模型、指标与对象关系——以及哪些口径必须由人确认

Schema

表、列、主键、外键候选

历史 SQL

报表与看板里真实在跑的口径

指标平台

已有指标定义与命名

业务文档

术语、对象、流程描述

抽取 → 校验 → 合并

- LLM 从 SQL / 报表反推候选 dataset、关系、指标
- dosi validate 对规范校验
- 相似度分析，合并近似重复的指标
- 版本化：每次变更可 diff、可回滚
- 产物：model.yaml + ontology.yaml

案例语义模型

```
datasets: orders · order_items
          customers · products
          channels
metrics:  revenue · order_count
          avg_order_value
          cancellation_rate
concepts: Customer · Product
          Channel · Order
```

orders-200k 示例数据集，191K 订单，2024 全年

需要人工确认的业务口径

取消订单是否计入 revenue？退款在哪个日期入账？跨币种按下单日汇率还是结算日？「新客」的定义窗口是 30 天还是首单？—— 这些不是抽取问题，是决策问题；语义层的价值是让决策只做一次、写进定义、处处生效。

完整分析链路：「收入为什么下降？」

从趋势查询到归因与验证，每一步的真实工具调用、关键结果与 SQL 要点

#	步骤	工具调用	关键结果	语义层做了什么
1	**趋势查询**	`query(revenue, order_date:quarter)`	Q1 $9.72M → Q2 $7.97M（−18.0%）	编译 1 条 SQL，cancelled 过滤来自指标定义
2	**维度拆解**	`query(revenue, [region, category])` `query(revenue, customer_count, region)`	West −$1.25M；Electronics −$0.89M	订单与客户是两个粒度 → 拆成 2 个 CTE，避免扇出（手写 join 计数会多算 13 倍）
3	**明细检查**	`select(orders.*, region='west', status='cancelled')`	4–6 月取消率持续 ≈28%；全是老客，不是新客驱动	沿用同一关联路径，明细与指标对得上
4	**归因**	`attribute(revenue, [region, category])`	West 贡献 71.5% · 取消率 8.2% → 27.9%	term_wise 精确拆分，各段之和 = −$1.75M
5	**验证**	`explain + query` 对账	两个维度的分段合计都等于总变化（残差 < 1e-7）	返回 SQL 与定义，结果可独立复算

7 次
工具调用（含 1 次 list；Agent 实测 6–11 次）

0 条
手写 SQL

0 次
原样重试

Σ = 总变化
归因分段对账通过

数据为 orders-200k 示例集（191K 订单，2024 上半年），dosi 0.1.12 于 2026-10-13 复跑。

三条实现路径对照：看不见的错误，更少的 Token 和更快的速度

直接生成 SQL · 仅指标文本 prompt · 完整 Semantic Runtime。同一个五步任务

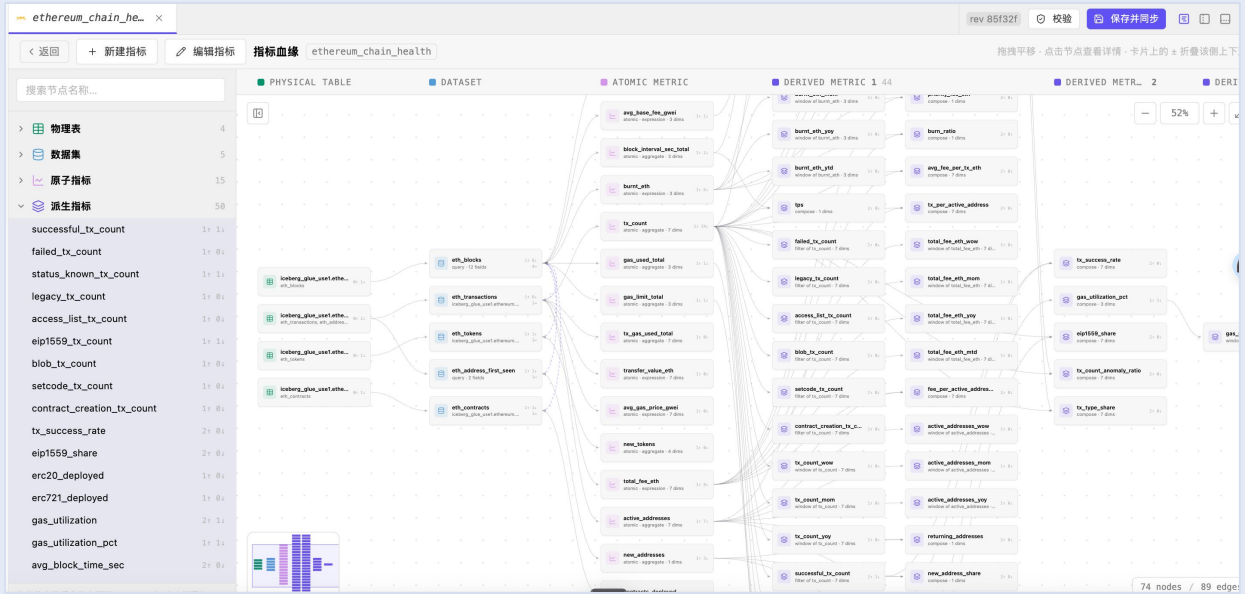
	直接生成 SQL	仅指标查询接口	完整 Semantic Runtime
1 **趋势查询**	✓ 3/3（先探查 status，自己推断口径）	✓ 3/3	✓ 3/3（口径来自定义）
2 **维度拆解**	✓ 3/3	✗ 1/3 时间窗漏了最后一天，静默出错	✓ 3/3
3 **明细检查**	✗ 2/3 cancelled 过滤丢失	✓ 3/3（回到手写 SQL）	✓ 3/3
4 **归因**	✓ 3/3（手写 SQL 拆分）	✓ 3/3（手写 SQL 拆分）	✓ 3/3（attribute，自带对账）
5 **验证**	✓ 自己对账	✓ 自己对账	✓ 引擎对账 + SQL 可复算
错误被发现的时机	从未：明细结果错误	从未：错数直接进了报告	编译期，结构化报错带 candidates，一次修正
手写 SQL	29–74 条	13–18 条	0 条
工具调用 / 耗时	5–7 次 / 46 s	5–9 次 / 39 s	6–9 次 / 38 s

n/3 = 同一任务 3 次运行中该步正确的次数 ✓ 全对 ✗ 有错 | 手写 SQL 与调用次数为 3 次运行的范围

差别不在「能不能跑出数」，而在错误是否被发现：前两条路径的错数静默进入结果，完整 Runtime 在编译期暴露并一次修正

指标体系是数据的压缩：少量原子指标，组合出深度归因

ethereum_chain_health：4 张物理表 → 5 个数据集 → 15 个原子指标 → 50 个派生指标（74 个节点 / 89 条边）



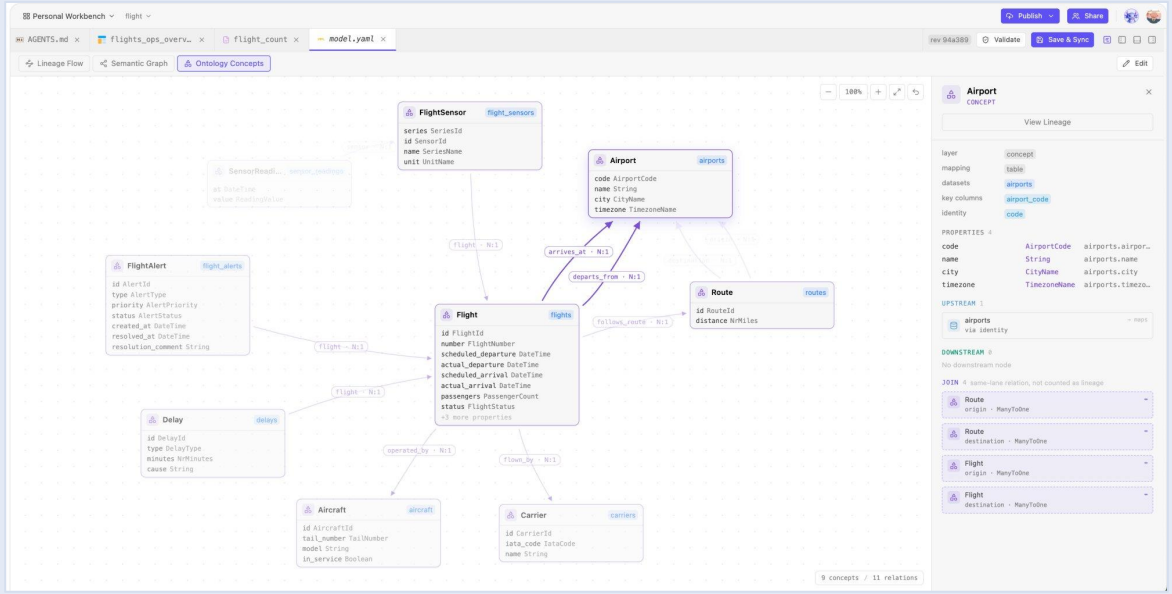
复杂指标体系能做什么

- 深度归因：沿血缘逐层下钻，从派生指标一路追到原子指标与物理表
- 简化物理表：4 张表承载 65 个指标，指标不再各自绑表、各写一遍 SQL
- 数据的压缩：15 个原子指标 + 组合规则，推出 50 个派生指标

复杂的指标体系不是负担：用少量原子指标和组合规则表达业务口径，并支持沿血缘的深度归因

Ontology：构建在语义层之上，提升确定性

Apache Ossie 的 Ontology 建在语义层之上；示例 flight：9 个概念、11 条关系，lowering 到语义模型执行



Ontology 提升确定性

- Concept mapping：概念绑定数据集与主键，属性映射到列
- Derived rules：在概念之上声明派生规则，业务含义不靠 prompt 猜
- Relationship：关系有名字、方向与多重性（N:1），路径是声明的而非猜测的

语义层保证计算正确，Ontology 在其上补足确定的业务含义：概念映射、派生规则与关系

总结：AI Native Semantic Layer 的三句话

01

语义层是Data agent的共识，语义层的质量决定了Data agent的下限

Agent 不会消除口径漂移，只会放大它。统一定义是 Day 1 的事；但只覆盖「查指标」的语义层，撑不住「解释变化」。

02

语义层编译正确的SQL，Agent负责检索，调用和发现

开放规范（Apache Ossie）+ 确定性的编译器 / 规划器（Dosi）：扇出、路径、粒度、方言由引擎证明，不由 prompt 提醒；少而可组合的指标，而不是几千个 + RAG。

03

本体是更确定性的 RAG，是语义层的导航

query / select / attribute / describe / explain 共用一套语义；结构化错误让 Agent 修正而不是重试；本体改变的只是 Agent 写查询前看到什么（业务语义的映射，确定的值域，requires描述的规则，可选的路径，跨语义模型的关系）

dosi.datus.ai （Apache Ossie SQL compiler） · Apache Ossie （语义层的iceberg moment，指标定义最大公约数）

《Apache Ossie + Dosi: A 10-Minute Semantic Layer for Your AI Agent》· 《What Makes a Semantic Layer Truly AI-Native?》

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

THANKS

如何构建 AI Native 的数据语义层

Dosi: Any database. Any agent. An Apache Ossie-native semantic layer for metrics.



扫码加入Datus开源社区交流群